# 14 · Form Validation and Custom Validators

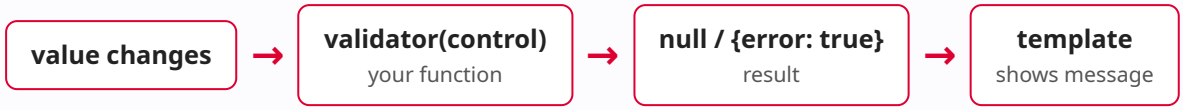
Intermediate  Angular learning notes

## a. Definition

A **validator** is a function that checks a control and returns `null` if the value is fine, or an **error object** (like `{ spaces: true }`) if it is not. Built-in validators cover common cases; for everything else you write your own.

A **cross-field validator** is attached to the whole `FormGroup` so it can compare several fields, for example *password* and *confirm password*. Errors then live on `form.errors` instead of on a single control.

## b. Diagram




Validators run whenever a value changes; errors are shown from `errors`

## c. Syntax

```
const noSpaces: ValidatorFn = (control) =>
  /\s/.test(control.value) ? { spaces: true } : null;

new FormGroup({ ... }, { validators: passwordsMatch });   // group-level

form.controls.username.errors?.['spaces']
form.errors?.['mismatch']
```

## d. Example Program

`Programs/14_Form_Validation/signup-validators.component.ts`

```
import { Component } from '@angular/core';
import { AbstractControl, FormControl, FormGroup, ReactiveFormsModule, ValidationErrors,
ValidatorFn, Validators } from '@angular/forms';

// 1. Custom validator for ONE control: returns null (valid) or an error object
const noSpaces: ValidatorFn = (control: AbstractControl): ValidationErrors | null =>
  /\s/.test(control.value) ? { spaces: true } : null;

// 2. Cross-field validator for the WHOLE group
const passwordsMatch: ValidatorFn = (group: AbstractControl): ValidationErrors | null => {
  const password = group.get('password')?.value;
  const confirm = group.get('confirm')?.value;
  return password === confirm ? null : { mismatch: true };
};

@Component({
  selector: 'app-root',
  imports: [ReactiveFormsModule],
  template: `
```

```
    <form [formGroup]="form">
      <input id="username" formControlName="username" placeholder="Username" />
      @if (form.controls.username.errors?.['spaces']) { <small>No spaces allowed.</small> }

      <input id="password" type="password" formControlName="password" placeholder="Password"
/>
      @if (form.controls.password.errors?.['minlength']) { <small>Minimum 6 characters.
</small> }

      <input id="confirm" type="password" formControlName="confirm" placeholder="Confirm" />
      @if (form.errors?.['mismatch'] && form.controls.confirm.dirty) { <small>Passwords do not
match.</small> }
    </form>
    <p>{{ form.valid ? 'Form is valid' : 'Form is invalid' }}</p>
  `,
})
export class SignupValidatorsComponent {
  form = new FormGroup(
    {
      username: new FormControl('', { nonNullable: true, validators: [Validators.required,
noSpaces] }),
      password: new FormControl('', { nonNullable: true, validators: [Validators.required,
Validators.minLength(6)] }),
      confirm: new FormControl('', { nonNullable: true }),
    },
    { validators: passwordsMatch },
  );
}
```

## e. Output

Rendered in the browser (text on screen, after the UI steps below):

1. Type a username containing a space.
2. Use a valid username but two different passwords.
3. Make the passwords match.

```
--- Space in username ---
No spaces allowed.
Form is invalid

--- Passwords differ ---
Passwords do not match.
Form is invalid

--- All valid ---
Form is valid
```

## f. Real-World Example (English + தமிழ்)

**English**

When opening a bank account you type the password twice. A rule that compares both fields tells you 'passwords do not match'. That is a cross-field validator.

**தமிழ்**

வங்கியில் புதிய கணக்குத் தொடங்கும்போது, கடவுச்சொல்லை இரண்டு முறை எழுதச் சொல்வார்கள். இரண்டும் ஒன்றாக இல்லையென்றால் 'பொருந்தவில்லை' என்று சொல்லி மீண்டும் எழுதச் சொல்வார்கள். ஒரே கட்டத்தை மட்டும் பார்த்து இதை முடிவு செய்ய முடியாது - இரண்டு கட்டங்களையும் ஒப்பிட வேண்டும்.

தனிக் கட்டத்துக்கான விதி (இடைவெளி கூடாது) = custom validator. இரண்டு கட்டங்களை ஒப்பிடும் விதி = cross-field validator; அது முழு `FormGroup`-இல் இணைக்கப்படும்.

## g. Pros

- Reusable rules
- Cross-field checks
- Async validators for server checks

## h. Cons

- Group errors are easy to forget to display
- Complex validators need good tests

## i. Advanced / New Features

- Typed validators with typed forms
- Schema based validation in experimental Signal Forms

## j. Common Mistakes

- Returning `false` or `true` from a validator; it must return `null` or an error object.
- Putting a cross-field validator on one control, where it cannot see the other control.
- Showing all errors at once; show the first relevant message per field.
- Forgetting that group validators set `form.errors`, not the control's `errors`.

## k. Interview Questions

**Q1. What does a validator function return?**
`null` when valid, otherwise an object describing the error, e.g. `{ spaces: true }`.

**Q2. How do you validate two fields together?**
Attach a validator to the parent `FormGroup` that reads both controls.

**Q3. What is an async validator?**
A validator returning an Observable/Promise (e.g. check username availability on the server).

**Q4. How do you read errors in the template?**
`control.errors?.['key']` for a control, or `form.errors?.['key']` for a group.

## I. Practice Exercises

1. Write a `strongPassword` validator that needs a digit and an uppercase letter.
2. Add a cross-field validator ensuring `endDate` is after `startDate`.
3. Create a validator factory `forbiddenWord(word)` that rejects a given word in a comment box.